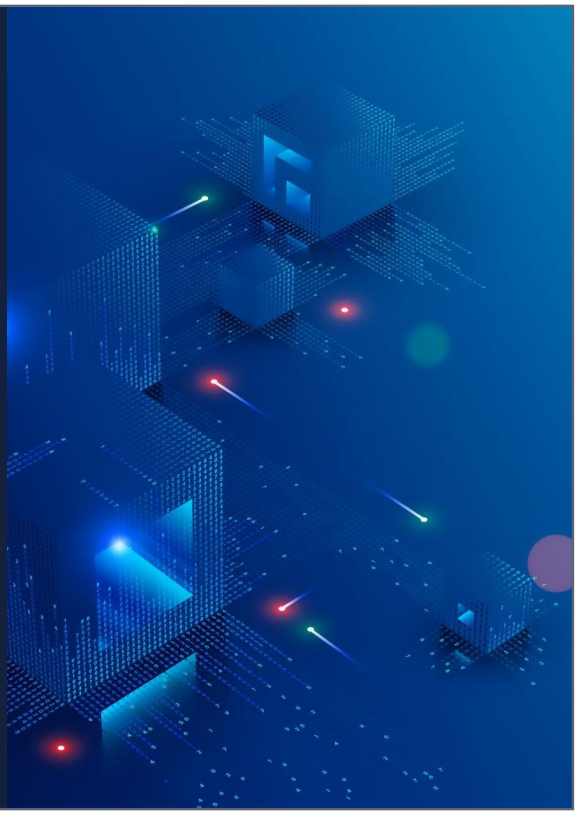
MODULE 09

Event-Driven Architecture

Commands, events and queries; event sourcing, CQRS and message brokers in Python

```
>>> bus.publish(OrderPlaced(order_id=17))
```



AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering

Learning Objectives

By the end of this module, you will be able to:

1. Classify messages as commands, events or queries and name them accordingly
2. Compare request-driven and event-driven collaboration and their coupling
3. Explain event notification, event-carried state transfer, event sourcing and CQRS
4. Implement an event-sourced model in Python: append-only store, replay, snapshots
5. Build a small CQRS example with separate command and query models
6. Publish and consume messages with RabbitMQ (`pika`) and Kafka (`confluent-kafka`)
7. Design reliable consumers with delivery guarantees, idempotency and the outbox
8. Choose between Kafka and RabbitMQ for a given workload

MODULE 09 · EVENT-DRIVEN ARCHITECTURE

Module Roadmap

01 **Collaboration: Commands, Events and Queries**
How distributed services talk to each other

02 **Event Notification**
Tell others that something happened — and nothing more

03 **Event-Carried State Transfer**
Put the data into the event and keep a local copy

04 **Event Sourcing**
Store every change as an event; derive the state by replaying

05 **CQRS**
Command Query Responsibility Segregation: separate models for writing and reading

06 **Messaging in Practice**
From in-memory event buses to RabbitMQ and Kafka

07 **Apache Kafka vs RabbitMQ**
Two brokers, two philosophies: a replayable log vs a smart router

01

Collaboration: Commands, Events and Queries

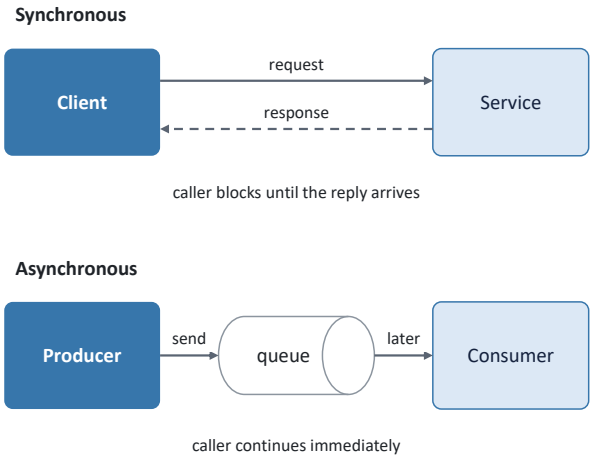
How distributed services talk to each other

- Synchronous vs asynchronous collaboration
- Commands couple, events decouple
- Commands, events and queries compared
- Request-driven vs event-driven order management

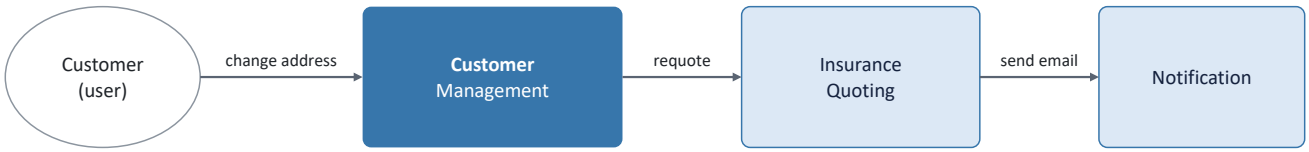
How Distributed Services Collaborate

Style	Caller waits?	Receivers
Request/response	yes (synchronous)	exactly one
Fire-and-forget	no	exactly one
Multicast	no	a known group
Push notification	no	client devices
Publish/subscribe	no	any subscriber

> **Note**
> Microservices and SOA (Module 6) are distributed architectures: their services must collaborate over the network.



Running Example: A Customer Changes Address



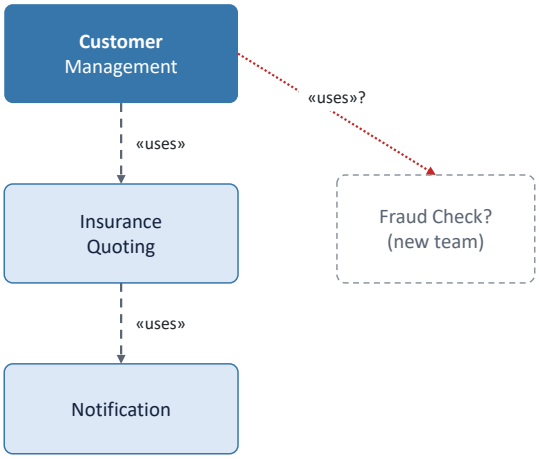
1 — change address
the user asks Customer Management to update the address

2 — requote
Customer Management tells Insurance Quoting to recalculate

3 — send email
Insurance Quoting tells Notification to inform the user

> **Key idea**
> Every arrow is a **command**: an imperative request sent to one specific receiver.

Commands Couple the Sender to the Receiver

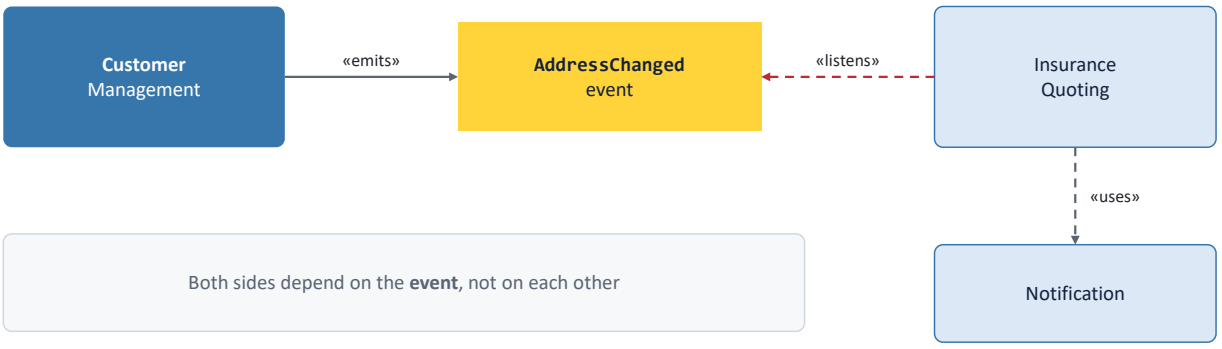


- The sender **knows** every receiver and its API
- A new consumer means **changing the sender**
- Receiver down or slow ⇒ the sender's call fails or waits
- Upside: the whole flow is visible in one place

Watch out

Dependencies point from the **decider** to the **doer**: Customer Management must change whenever someone else needs to react.

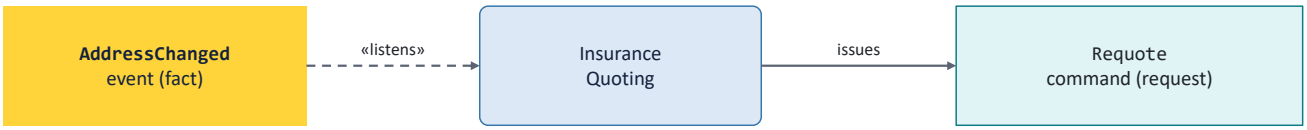
Announcing a Fact: The AddressChanged Event



Key idea

Event Notification (Fowler, 2017): a service announces that something happened and does not care who reacts.

Commands, Events and Queries Compared

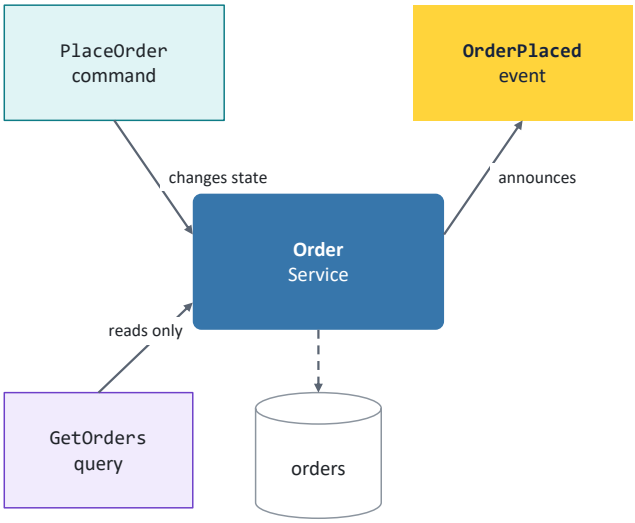


	State change	Response	Receivers	Naming
Command	requested to happen	maybe (accept/reject)	exactly one	imperative: `ChangeAddress`
Event	has already happened	never	zero or more	past tense: `AddressChanged`
Query	none	always	exactly one	question: `GetCustomer`

Key idea

A command can be rejected; an event cannot — it is already history.

One Service, Three Kinds of Messages



messages.py

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class PlaceOrder:          # command
    customer: str
    item: str


@dataclass(frozen=True)
class OrderPlaced:         # event
    order_id: int
    item: str


@dataclass(frozen=True)
class GetOrders:           # query
    customer: str
```

QUICK CHECK

Command, Event or Query?

Classify each message. Justify your answer from its name and meaning.

1. `ReserveSeat(flight="TK1", seat="12A")`
2. `SeatReserved(flight="TK1", seat="12A")`
3. `GetAvailableSeats(flight="TK1")`
4. `PaymentFailed(order_id=7)`
5. `CancelBooking(booking_id=31)`
6. `FindBookingsByCustomer(customer_id=42)`

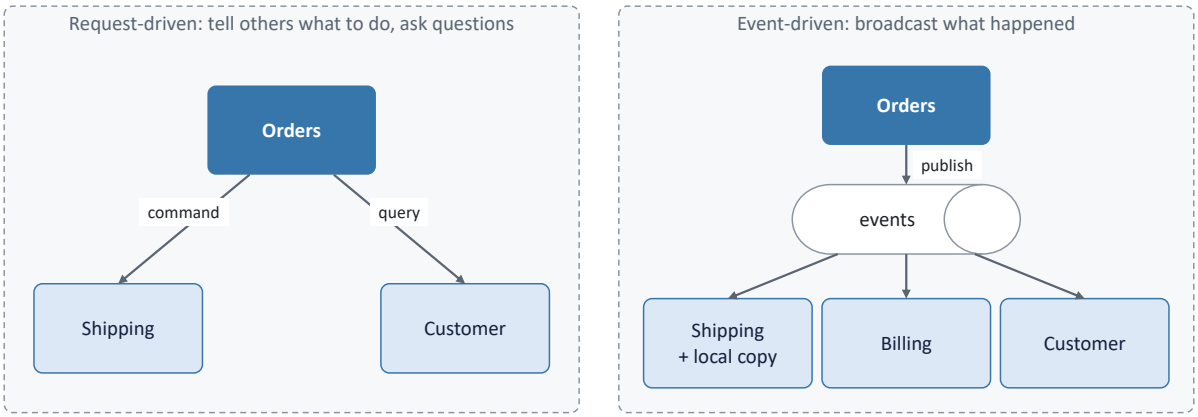
✓ Ask: does it **request** a change, **announce** a fact, or **ask** for data?

ANSWER

Command, Event or Query?

Message	Kind	Why
`ReserveSeat`	command	imperative; may be rejected (seat taken)
`SeatReserved`	event	past tense; the reservation already happened
`GetAvailableSeats`	query	asks for data, changes nothing
`PaymentFailed`	event	a failure is also a fact that happened
`CancelBooking`	command	requests a change; may be refused (too late)
`FindBookingsByCustomer`	query	question; always returns a (possibly empty) list

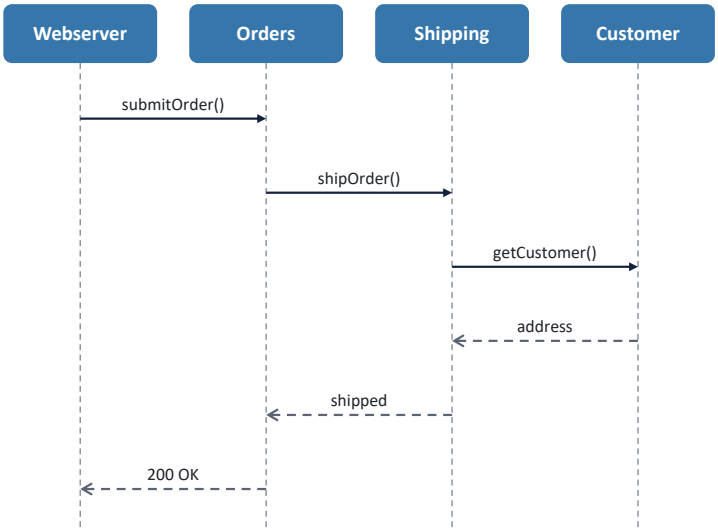
Request-Driven vs Event-Driven Thinking



Key idea

Request-driven: the caller decides what happens next. Event-driven: each receiver decides for itself and keeps the data it needs.

A Request-Driven Order Management System

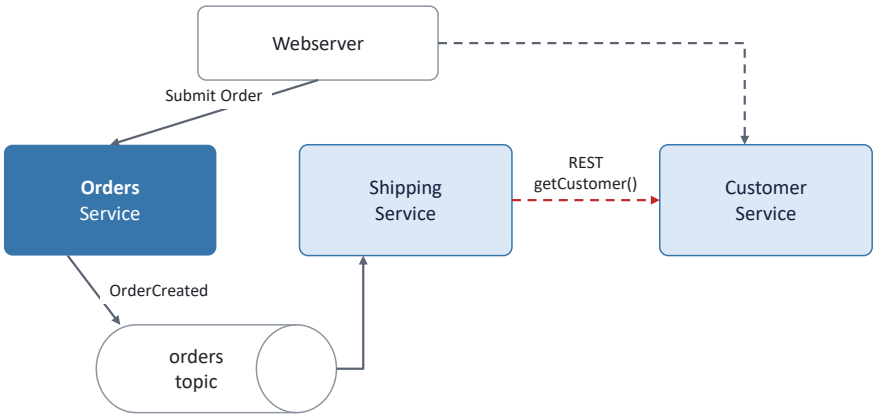


- Orders **commands** Shipping: `shipOrder()`
- Shipping **queries** Customer: `getCustomer()`
- Each call is **synchronous** (REST, Module 8)
- Latencies add up along the chain
- All services must be up at the same time

Watch out

Temporal coupling: one slow service slows the whole request.

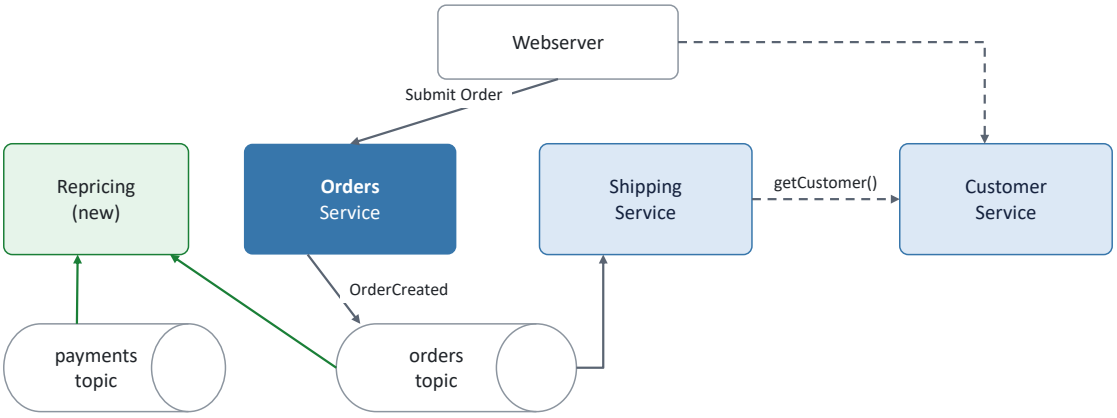
An Event-Driven Order Management System



Key idea

Orders no longer knows that Shipping exists. Shipping still **queries** Customer: a synchronous call-back remains.

Extensibility: Adding a Service Without Changes



Tip

Repricing plugs in by subscribing to existing topics — no other service changes (open for extension, closed for modification).

QUIZ

Quick Check

A new Fraud Check service must react to every `OrderCreated` event. In the event-driven design, what has to change?

- **A** Orders must call Fraud Check after saving an order
- **B** Only the new service: it subscribes to the orders topic
- **C** The webserver must forward each order to Fraud Check
- **D** Shipping must forward `OrderCreated` to Fraud Check

ANSWER

Quick Check

A new Fraud Check service must react to every `OrderCreated` event. In the event-driven design, what has to change?

- A Orders must call Fraud Check after saving an order
- **B** Only the new service: it subscribes to the orders topic
- C The webserver must forward each order to Fraud Check
- D Shipping must forward `OrderCreated` to Fraud Check

★ **Why**

Producers do not know their consumers. A new consumer subscribes to the existing topic; Orders, Shipping and the webserver stay unchanged.

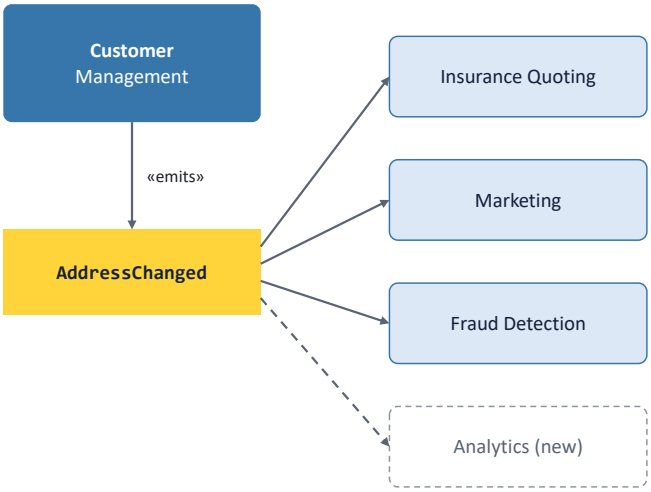
02

Event Notification

Tell others that something happened — and nothing more

- Thin events and the call-back problem
- Event notification in Python
- Decoupling vs visibility of the overall flow

The Event Notification Pattern

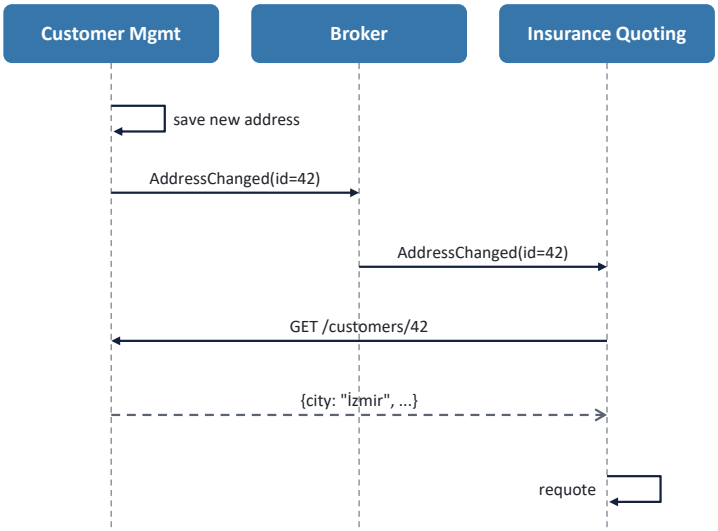


- A service **notifies** others that its state changed
- The event is **thin**: type, id, timestamp
- Sender does not know or care who listens
- Listeners subscribe without changing the sender
- Sender does not expect any response

Key idea

Decoupling is the point: teams can add listeners independently.

Thin Events Often Need a Call-Back



- The event says *what* happened, not the new data
- Listeners **query back** for details
- Call-backs bring back runtime coupling
- Load on the sender grows with every listener

> **Note**
> Section 3 removes the call-back by putting the data into the event.

Event Notification in Python

notification.py

```python
from dataclasses import dataclass

customers = {42: {"name": "Ada", "city": "Ankara"}}  # owned by CM


@dataclass(frozen=True)
class AddressChanged:
    customer_id: int                    # thin: no new data


def requote(event: AddressChanged) -> None:
    city = customers[event.customer_id]["city"]   # call-back
    print(f"requote customer {event.customer_id} in {city}")


listeners = [requote, lambda e: print("email", e.customer_id)]
customers[42]["city"] = "İzmir"                   # change ...
for listener in listeners:                        # ... notify
    listener(AddressChanged(customer_id=42))
```

OUTPUT

```
requote customer 42 in İzmir
email 42
```

2 · EVENT NOTIFICATION

Event Notification: Strengths and Weaknesses

✓ Strengths

- Decouples receivers from the sender
- New listeners need no sender changes
- Sender stays simple and fast
- Tiny, stable event schema

✗ Weaknesses

- No statement of the overall behaviour
- Flow is spread across many listeners
- Thin events cause call-back queries
- Harder to debug and test end to end

Tip

Make the hidden flow visible: correlation IDs on every event, distributed tracing (e.g. OpenTelemetry) and an event catalogue.

QUIZ

Quick Check

What is the main weakness of the Event Notification pattern?

A. The sender must know every receiver

B. The overall business flow is hard to see and debug

C. Events cannot be delivered asynchronously

D. Receivers cannot react independently

ANSWER

Quick Check

What is the main weakness of the Event Notification pattern?

- A The sender must know every receiver
- **B The overall business flow is hard to see and debug**
- C Events cannot be delivered asynchronously
- D Receivers cannot react independently

Why

Behaviour emerges from independent listeners, so no single place describes the whole process. Options A, C and D describe the opposite of event notification.

03

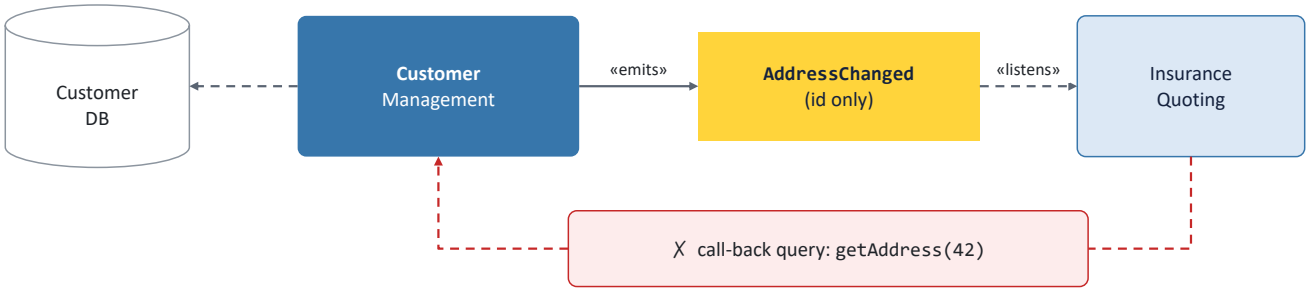
Event-Carried State Transfer

Put the data into the event and keep a local copy

- Why call-backs hurt
- Fat events and local replicas in Python
- Eventual consistency
- Thin vs fat events

3 · EVENT-CARRIED STATE TRANSFER

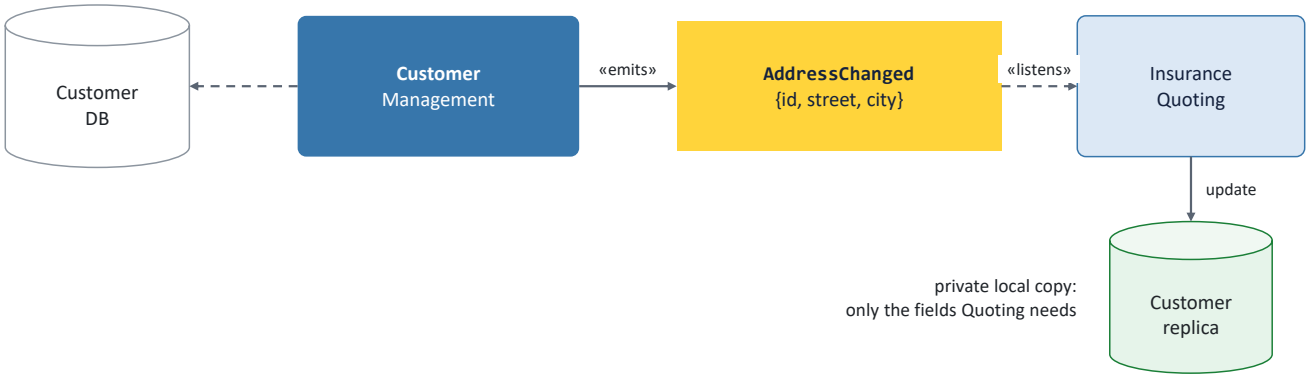
The Call-Back Problem



- Customer data lives in **one** database, owned by Customer Management
- Every notification triggers a query back to the owner
- The owner must be **up** and fast whenever any listener reacts
- Load on Customer Management grows with the number of listeners

3 · EVENT-CARRIED STATE TRANSFER

State Transfer: Events That Carry the Data



Key idea

Event-carried state transfer (Fowler, 2017): events carry the changed data, so receivers keep a local copy and never call back.

3 · EVENT-CARRIED STATE TRANSFER

Fat Events and a Local Replica in Python

state_transfer.py

```python
from dataclasses import dataclass, field


@dataclass(frozen=True)
class AddressChanged:
    customer_id: int
    city: str                          # fat: carries the new state


@dataclass
class InsuranceQuoting:
    cities: dict[int, str] = field(default_factory=dict)  # replica

    def on_address_changed(self, event: AddressChanged) -> None:
        self.cities[event.customer_id] = event.city

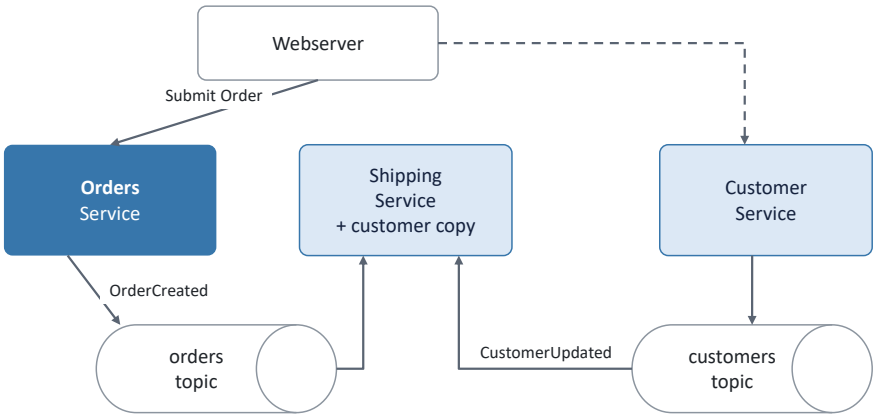

quoting = InsuranceQuoting()
quoting.on_address_changed(AddressChanged(42, "İzmir"))
print("quote uses", quoting.cities[42])       # no call-back
```

OUTPUT

```
quote uses İzmir
```


3 · EVENT-CARRIED STATE TRANSFER

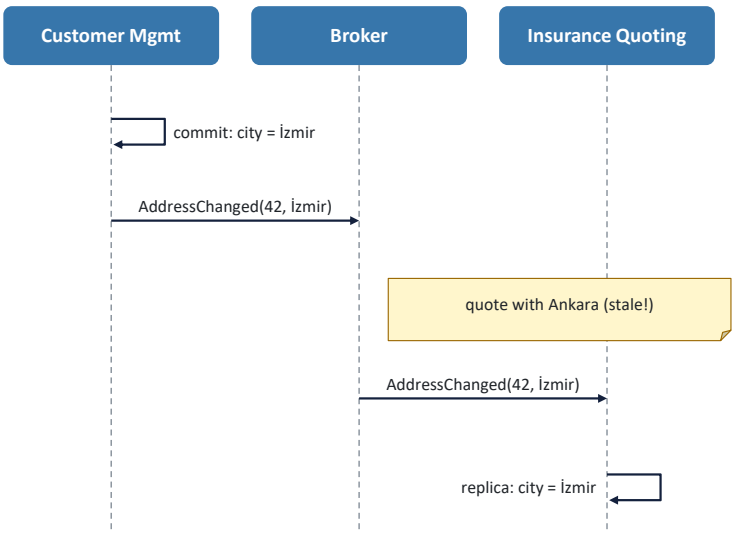
Replicating Customer Data in the Order System



Key idea

No REST call remains: Shipping answers `getCustomer()` from its own copy, filled by `CustomerUpdated` events.

Eventual Consistency



- The replica lags behind the owner for a short **window**
- Reads in that window see **stale** data
- Without new changes, all copies **converge**
- Design business rules and UI for the delay

Watch out

Usually milliseconds to seconds — but unbounded if a consumer is down.

Thin vs Fat Events: The Trade-Offs

	Event notification (thin)	Event-carried state transfer (fat)
Payload	type, id, timestamp	the changed data
Call-back to the owner	often needed	not needed
Works when owner is down	no (if it calls back)	yes, from the local replica
Load on the owner	grows with listeners	reduced
Data duplication	none	replicated data ✗
Consistency	reads the latest state	eventual consistency ✗
Schema coupling	small	consumers depend on the payload

Key idea

Decoupling ✓ and reduced load on the supplier ✓ are paid for with replicated data ✗ and eventual consistency ✗.

QUIZ

Quick Check

Insurance Quoting must keep producing quotes while Customer Management is down for maintenance. Which collaboration fits best?

- **A** Event notification with a call-back query
- **B** Event-carried state transfer with a local replica
- **C** A synchronous REST query for every quote
- **D** Customer Management sends a `Requote` command

ANSWER

Quick Check

Insurance Quoting must keep producing quotes while Customer Management is down for maintenance. Which collaboration fits best?

- A Event notification with a call-back query
- **B** Event-carried state transfer with a local replica
- C A synchronous REST query for every quote
- D Customer Management sends a `Requote` command

★ **Why**

Only a local replica, filled from fat events, lets Quoting work without the owner. A, C and D all need Customer Management at the moment of quoting.

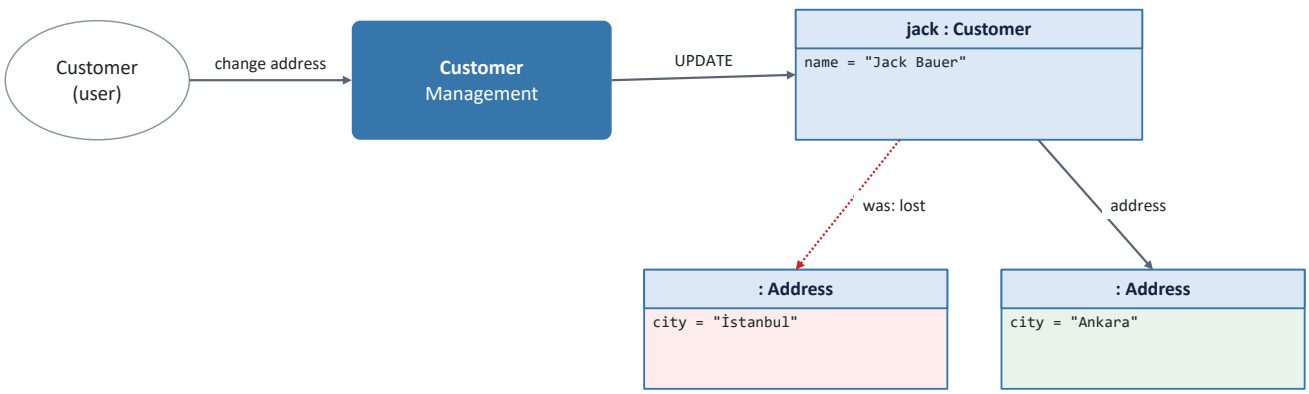
04

Event Sourcing

Store every change as an event; derive the state by replaying

- Current state vs history of changes
- An append-only event store in Python
- Replaying, historic state and snapshots
- Benefits, costs and event versioning

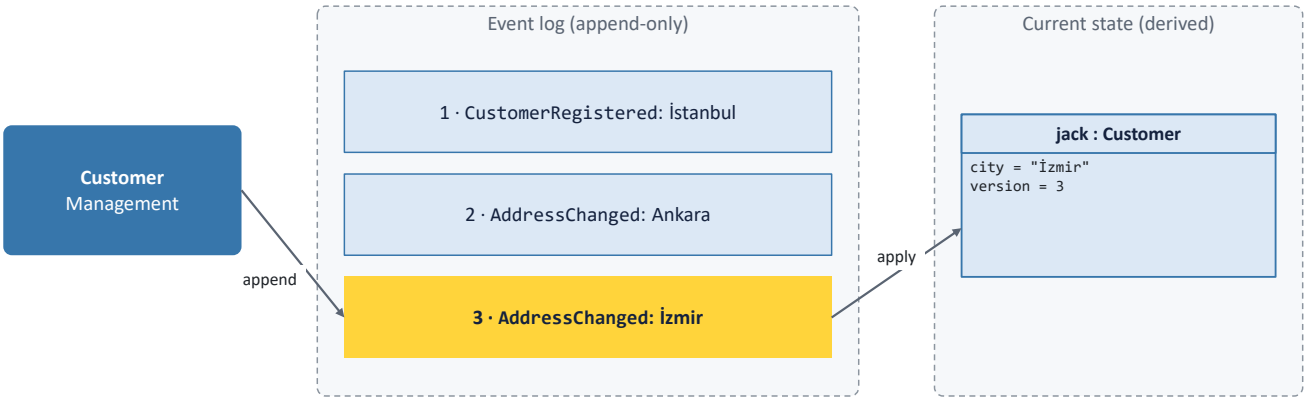
Current State Overwrites the Past



Watch out

The database knows **where** Jack lives now — not where he lived before, **when** he moved or **why**.

Event Sourcing: Store the Changes, Not the State



Key idea

Event sourcing (Fowler, 2005): capture every change to the application state as an event and store the events in order. The log is the source of truth.

Events and an Append-Only Event Store

events.py

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class CustomerRegistered:
    customer_id: str
    city: str


@dataclass(frozen=True)
class AddressChanged:
    customer_id: str
    city: str


Event = CustomerRegistered | AddressChanged
```

event_store.py

```python
class EventStore:
    """Append-only, in-memory event log."""

    def __init__(self) -> None:
        self._log: dict[str, list[Event]] = {}

    def append(self, stream: str, event: Event) -> int:
        events = self._log.setdefault(stream, [])
        events.append(event)
        return len(events)          # new version

    def read(self, stream: str) -> tuple[Event, ...]:
        return tuple(self._log.get(stream, []))
```

Key idea

No `update` or `delete`: the store can only **append** and **read**.

4 · EVENT SOURCING

Rebuilding State by Replaying Events

customer.py

```python
@dataclass
class Customer:
    customer_id: str = ""
    city: str = ""
    version: int = 0

    def apply(self, event: Event) -> None:
        match event:
            case CustomerRegistered(cid, city):
                self.customer_id = cid
                self.city = city
            case AddressChanged(_, city):
                self.city = city
        self.version += 1
```

```python
from collections.abc import Iterable


def replay(events: Iterable[Event]) -> Customer:
    """Rebuild the state from scratch."""
    customer = Customer()
    for event in events:
        customer.apply(event)
    return customer
```

Key idea

The state is disposable: delete it and replay the log whenever you like.



4 · EVENT SOURCING

Current and Historic State from One Log

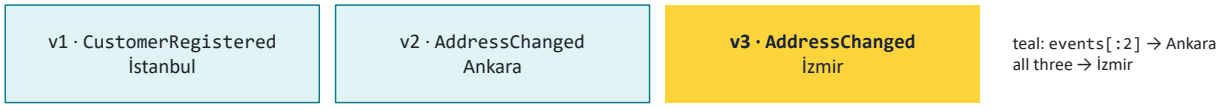
main.py

```python
store = EventStore()
store.append("c-42", CustomerRegistered("c-42", "İstanbul"))
store.append("c-42", AddressChanged("c-42", "Ankara"))
store.append("c-42", AddressChanged("c-42", "İzmir"))

events = store.read("c-42")
print(replay(events))                  # current state
print(replay(events[:2]))              # historic state: version 2
print(len(events), "events stored, none overwritten")
```

OUTPUT

```
Customer(customer_id='c-42', city='İzmir', version=3)
Customer(customer_id='c-42', city='Ankara', version=2)
3 events stored, none overwritten
```



4 · EVENT SOURCING

Snapshots: Faster Rebuilds

snapshot.py

```python
from dataclasses import dataclass, replace


@dataclass(frozen=True)
class Snapshot:
    version: int                # last event included
    state: Customer


def load(events: tuple[Event, ...], snap: Snapshot) -> Customer:
    customer = replace(snap.state)    # copy: keep snapshot intact
    for event in events[snap.version:]:   # only the newer events
        customer.apply(event)
    return customer


snap = Snapshot(2, replay(events[:2]))      # saved earlier
print(load(events, snap))
```

OUTPUT

```
Customer(customer_id='c-42', city='İzmir', version=3)
```


TRACE THE CODE

Trace the Replay

`Deposited` and `Withdrawn` are frozen dataclasses with one field, `amount`.

```python
def replay(events: list[Deposited | Withdrawn]) -> int:
    balance = 0
    for event in events:
        match event:
            case Deposited(amount):
                balance += amount
            case Withdrawn(amount):
                balance -= amount
    return balance


log = [Deposited(100), Withdrawn(30), Deposited(50), Withdrawn(70)]
print(replay(log))
print(replay(log[:2]))
print(replay(log[1:]))
```

ANSWER

Trace the Replay

OUTPUT

```
50
70
-50
```

- Full log: 100 − 30 + 50 − 70 = **50** (current balance)
- `log[:2]`: state after version 2 = **70** (historic state)
- `log[1:]` drops the first event: **−50**, an impossible balance
- Replay needs the **complete** log, from the very first event

4 · EVENT SOURCING

Event Sourcing: Benefits and Costs

Benefit ✓	Why
Audit	every change is recorded, with its reason
Debugging	replay the events to reproduce a bug
Historic state	answer "what was true on 1 March?"
Memory image	keep state in RAM, rebuild on restart

Cost ✗	Why
Unfamiliar	a new programming model for the team
Event schema	events are a permanent contract
Identifiers	entities need stable ids from the start
Versioning	old events must stay readable forever

Tip

Use event sourcing where history is part of the domain (finance, logistics, compliance) — not for every CRUD table.

Evolving Events: Upcasting Old Versions

upcasting.py

```python
def upcast(raw: dict) -> dict:
    """Bring a stored event up to the current schema (v2)."""
    if raw.get("version", 1) == 2:
        return raw
    upgraded = dict(raw, version=2)       # never modify stored data
    address = upgraded.pop("address")     # v1: "street, city"
    upgraded["street"], upgraded["city"] = address.rsplit(", ", 1)
    return upgraded


v1 = {"type": "AddressChanged", "customer_id": "c-42",
      "address": "Kordon Blv. 1, İzmir"}
v2 = upcast(v1)
print(v2["version"], v2["street"], v2["city"], sep=" | ")
print("address" in v1, "address" in v2)
```

OUTPUT

```
2 | Kordon Blv. 1 | İzmir
True False
```

★ **Key idea**

Stored events are never rewritten. An **upcaster** converts old versions to the current schema while they are read.

QUIZ

Quick Check

Why do event-sourced systems take snapshots?

A. To delete old events and save disk space

B. To rebuild state faster by replaying only newer events

C. To allow events to be modified after they are stored

D. To replace the event store with a normal database

ANSWER

Quick Check

Why do event-sourced systems take snapshots?

- A. To delete old events and save disk space
- **B. To rebuild state faster by replaying only newer events**
- C. To allow events to be modified after they are stored
- D. To replace the event store with a normal database

★ **Why**

A snapshot caches the state at a known version; loading replays only the events after it. The log is still kept in full and never modified.

05

CQRS

Command Query Responsibility Segregation: separate models for writing and reading

- Commands, queries and responsibility segregation
- Command model vs query model
- CQRS with event sourcing
- A runnable CQRS example in Python
- When (not) to use CQRS

What Is CQRS?

CQRS separates the code that changes state (**commands**) from the code that reads it (**queries**), usually with a different model for each side.

Command side (write)

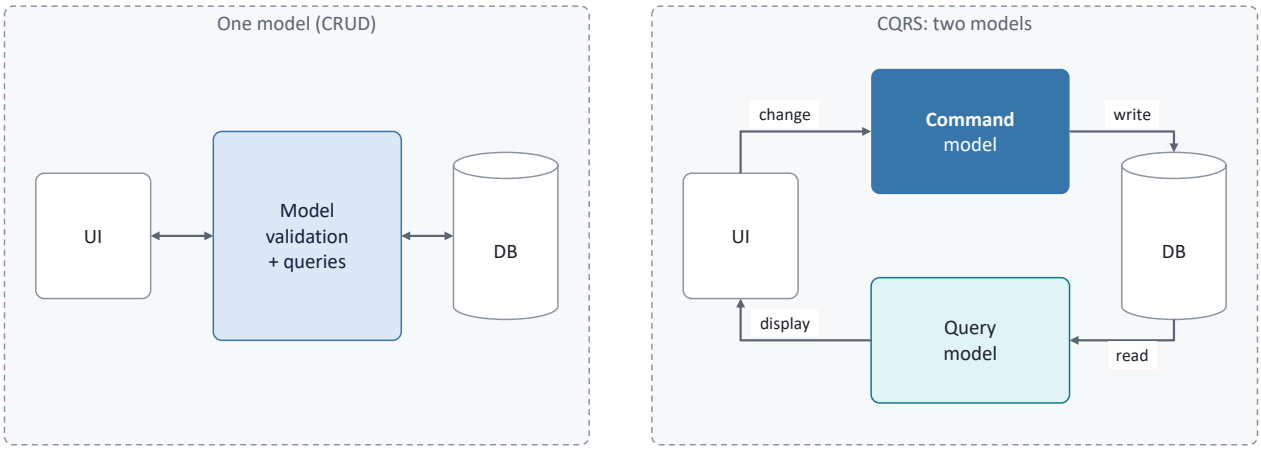
- Creates, updates, deletes
- Validates business rules
- Returns at most an id or an acknowledgement

Query side (read)

- Fetches lists, details, search results
- Never changes state
- Data shaped for the screen or report

> **Note**
> Origin: Meyer's command–query separation (CQS) for methods, applied to whole models (Young, 2010).

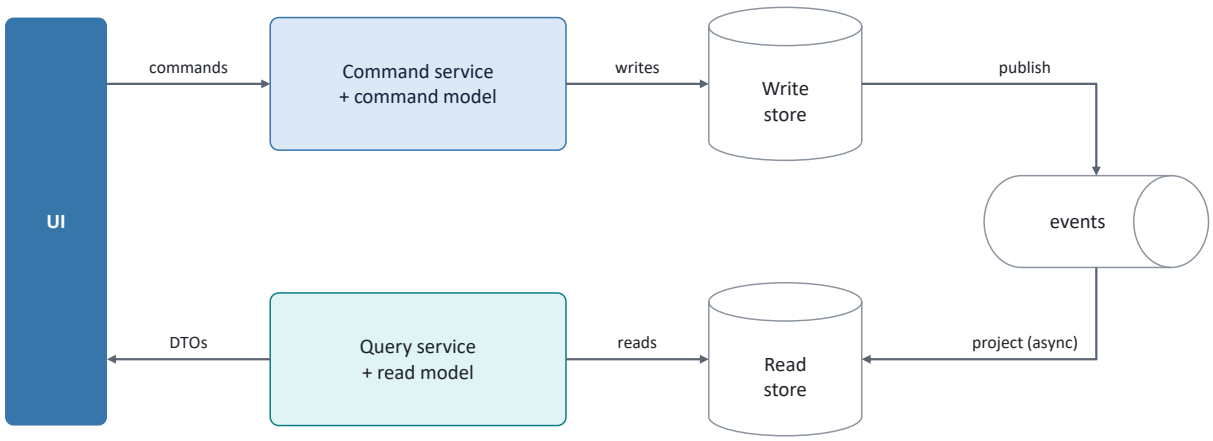
From One Model to Two



> **Key idea**
> Write model: rich domain rules. Read model: flat data shaped for each screen.

5 · CQRS

CQRS Architecture: Syncing the Read Side

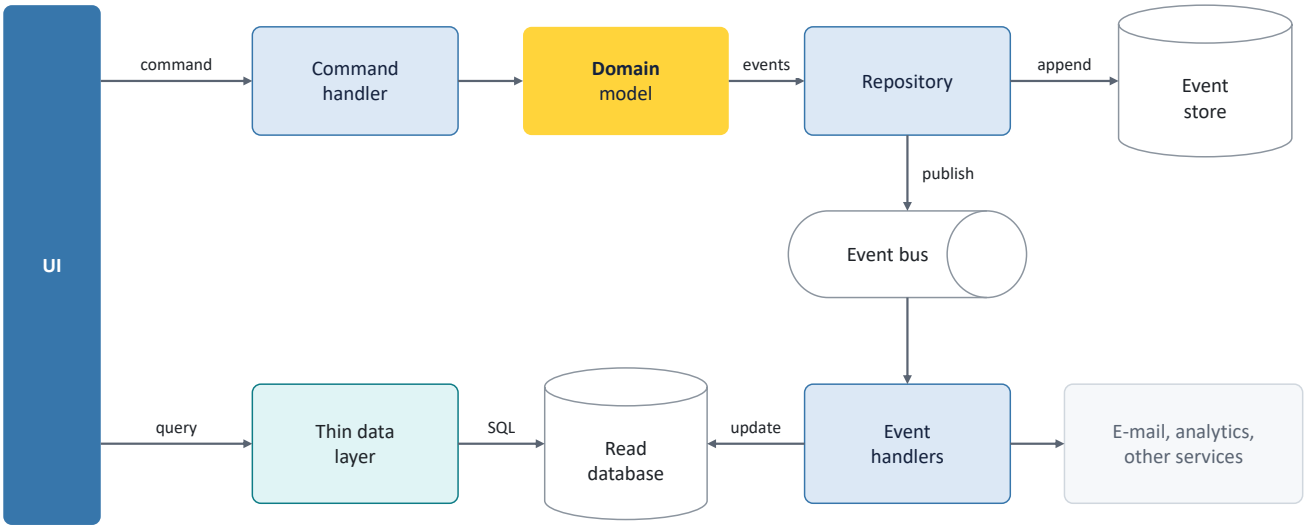


Watch out

The read side is updated **asynchronously**: scalable and fast, but eventually consistent.

5 · CQRS

CQRS with Event Sourcing



CQRS in Python: Commands, Events and Queries

messages.py

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class CreateOrder:                  # command
    customer_id: int
    product_id: int
    quantity: int


@dataclass(frozen=True)
class OrderCreated:                 # event
    order_id: int
    customer_id: int
    product_id: int
    quantity: int
```

```python
@dataclass(frozen=True)
class GetOrderDetails:              # query
    order_id: int


@dataclass(frozen=True)
class OrderDetails:                 # query result
    order_id: int
    customer_id: int
    product_id: int
    quantity: int
```

Tip

Frozen dataclasses replace the original hand-written `__init__` methods — and their `def__init__` typo.

CQRS in Python: The Command Side

event_bus.py

```python
from collections import defaultdict
from collections.abc import Callable


class EventBus:
    def __init__(self) -> None:
        self._handlers = defaultdict(list)

    def subscribe(self, event_type: type) -> Callable:
        def register(handler: Callable) -> Callable:
            self._handlers[event_type].append(handler)
            return handler
        return register

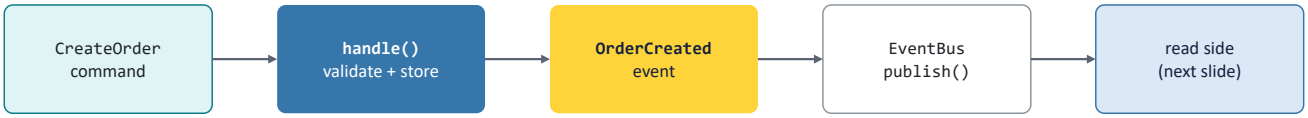
    def publish(self, event: object) -> None:
        for handler in self._handlers[type(event)]:
            handler(event)
```

write_side.py

```python
class CreateOrderHandler:
    """Write side: validate, store, publish."""

    def __init__(self, bus: EventBus) -> None:
        self.bus = bus
        self.orders: dict[int, CreateOrder] = {}

    def handle(self, command: CreateOrder) -> int:
        if command.quantity <= 0:
            raise ValueError("quantity must be positive")
        order_id = len(self.orders) + 1
        self.orders[order_id] = command
        self.bus.publish(OrderCreated(
            order_id, command.customer_id,
            command.product_id, command.quantity))
        return order_id
```



CQRS in Python: The Query Side

read_side.py

```python
bus = EventBus()
read_model: dict[int, OrderDetails] = {}       # query-side store


@bus.subscribe(OrderCreated)
def on_order_created(event: OrderCreated) -> None:
    """Projection: keep the read model up to date."""
    read_model[event.order_id] = OrderDetails(
        event.order_id, event.customer_id, event.product_id, event.quantity)


def get_order_details(query: GetOrderDetails) -> OrderDetails:
    return read_model[query.order_id]           # no business logic


orders = CreateOrderHandler(bus)
command = CreateOrder(customer_id=7, product_id=1001, quantity=2)
order_id = orders.handle(command)
print(get_order_details(GetOrderDetails(order_id)))
```

OUTPUT

```
OrderDetails(order_id=1, customer_id=7, product_id=1001, quantity=2)
```


CQRS: Advantages and Disadvantages

✓ Advantages

- Scale reads and writes independently
- Read models optimised for each query
- Simpler, focused command logic
- Security: writes guarded in one place

✗ Disadvantages

- More moving parts: two models, sync code
- Eventually consistent read side
- Duplicated data, projection bugs
- Harder to learn, debug and operate

Key idea

CQRS trades simplicity for flexibility — the trade only pays off in complex or high-load domains.

When to Use CQRS

Good fit

- Complex domain logic on the write side
- Very different read and write loads
- Many different views of the same data
- Already event-sourced

Poor fit

- Simple CRUD screens
- Strong read-after-write consistency needed everywhere
- Small team, small system

Watch out

Apply CQRS to specific **bounded contexts** (DDD), never to a whole system by default (Fowler, 2011).

QUIZ

Quick Check

A user places an order and immediately opens "My orders", but the new order is missing. A few seconds later it appears. What is the most likely cause?

A. The command was rejected by the command handler

B. The read model is updated asynchronously

C. The event store lost the `OrderCreated` event

D. The query handler modified the state

ANSWER

Quick Check

A user places an order and immediately opens "My orders", but the new order is missing. A few seconds later it appears. What is the most likely cause?

A. The command was rejected by the command handler

B. The read model is updated asynchronously

C. The event store lost the `OrderCreated` event

D. The query handler modified the state

Why

The projection runs after the command completes; until it has processed `OrderCreated`, the read model is stale. The order appears later, so nothing was lost.

06

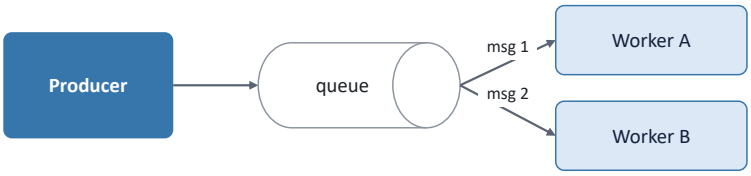
Messaging in Practice

From in-memory event buses to RabbitMQ and Kafka

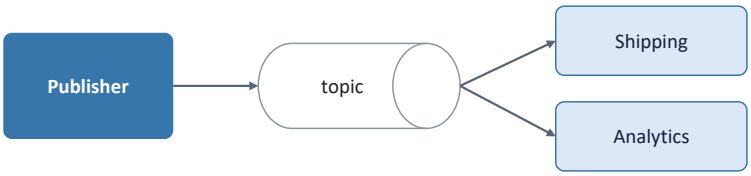
- An in-memory event bus and asyncio consumers
- RabbitMQ with `pika`: exchanges, queues, acknowledgements
- Kafka with `confluent-kafka`: topics, partitions, offsets
- Delivery guarantees, idempotent consumers, the outbox

Message Brokers: Queues and Topics

Queue (point-to-point): each message to **one** consumer



Topic (publish/subscribe): each subscriber gets a **copy**



A **message broker** adds:

- Decoupling in time (buffering)
- Durable storage of messages
- Routing to the right receivers
- Acknowledgements and redelivery
- Load balancing across workers

An In-Memory Event Bus in Python

event_bus.py

```python
from collections import defaultdict
from collections.abc import Callable


class EventBus:
    def __init__(self) -> None:
        self._handlers: dict[type, list[Callable]] = defaultdict(list)

    def subscribe(self, event_type: type, handler: Callable) -> None:
        self._handlers[event_type].append(handler)

    def publish(self, event: object) -> None:
        for cls in type(event).__mro__:           # base classes too
            for handler in self._handlers.get(cls, []):
                try:
                    handler(event)
                except Exception as exc:          # isolate failures
                    print(f"{handler.__name__} failed: {exc!r}")
```

Using the Event Bus

shop.py

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class OrderPlaced:
    order_id: int


def update_stock(event: OrderPlaced) -> None:
    raise ConnectionError("inventory service down")


bus = EventBus()
bus.subscribe(OrderPlaced, update_stock)
bus.subscribe(OrderPlaced, lambda e: print("email for order", e.order_id))
bus.subscribe(object, lambda e: print("audit:", e))      # every event
bus.publish(OrderPlaced(order_id=17))
```

OUTPUT

```
update_stock failed: ConnectionError('inventory service down')
email for order 17
audit: OrderPlaced(order_id=17)
```


Asynchronous Consumers with asyncio

async_consumers.py

```python
import asyncio


async def consumer(name: str, queue: asyncio.Queue[int | None]) -> None:
    while (order_id := await queue.get()) is not None:
        await asyncio.sleep(0.1)                # simulated I/O
        print(f"{name} shipped order {order_id}")

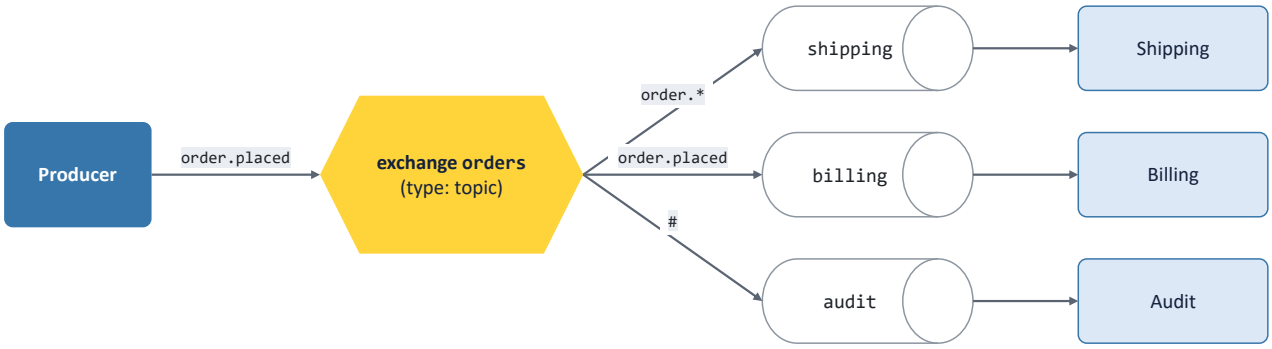

async def main() -> None:
    queue: asyncio.Queue[int | None] = asyncio.Queue()
    async with asyncio.TaskGroup() as group:
        for name in ("worker-1", "worker-2"):
            group.create_task(consumer(name, queue))
        for order_id in [101, 102, 103, 104, None, None]:
            await queue.put(order_id)           # None = stop


asyncio.run(main())
```

OUTPUT

```
worker-1 shipped order 101
worker-2 shipped order 102
worker-1 shipped order 103
worker-2 shipped order 104
```

RabbitMQ: Exchanges, Queues and Bindings



Exchange type	Delivers a message to every bound queue whose binding…
`direct`	key equals the routing key
`topic`	pattern matches: `*` = one word, `#` = zero or more words
`fanout`	exists — the routing key is ignored (broadcast)
`headers`	matches the message's header values

RabbitMQ Producer with pika

producer.py

```python
import json

import pika

params = pika.ConnectionParameters("localhost")
with pika.BlockingConnection(params) as connection:
    channel = connection.channel()
    channel.exchange_declare("orders", exchange_type="topic", durable=True)
    channel.confirm_delivery()            # broker confirms every publish
    event = {"type": "OrderPlaced", "order_id": 17, "total": 249.9}
    channel.basic_publish(
        exchange="orders",
        routing_key="order.placed",
        body=json.dumps(event),
        properties=pika.BasicProperties(
            content_type="application/json",
            delivery_mode=pika.DeliveryMode.Persistent),
    )
```

RabbitMQ Consumer with pika

consumer.py

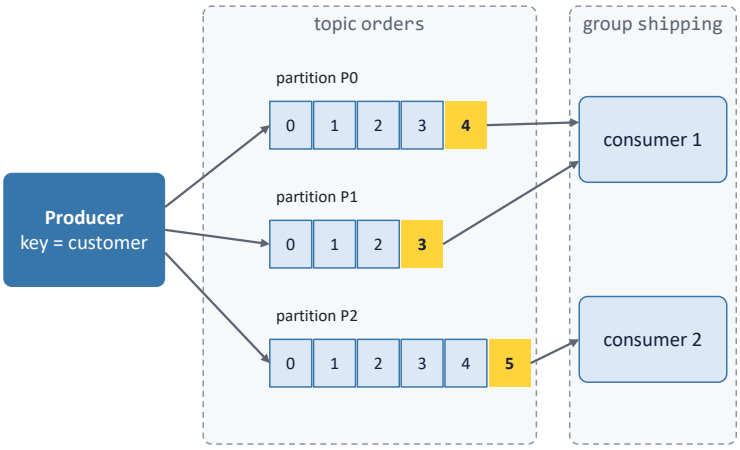
```python
import json

import pika


def on_message(channel, method, properties, body: bytes) -> None:
    event = json.loads(body)
    print("shipping order", event["order_id"])
    channel.basic_ack(delivery_tag=method.delivery_tag)  # after work


params = pika.ConnectionParameters("localhost")
with pika.BlockingConnection(params) as connection:
    channel = connection.channel()
    channel.exchange_declare("orders", exchange_type="topic", durable=True)
    channel.queue_declare("shipping", durable=True)
    channel.queue_bind("shipping", "orders", routing_key="order.*")
    channel.basic_qos(prefetch_count=10)       # max unacked messages
    channel.basic_consume("shipping", on_message)   # manual ack
    channel.start_consuming()
```


Kafka: Topics, Partitions and Offsets



- A **topic** is split into **partitions**
- Same key ⇒ same partition ⇒ ordered
- **Offset**: position within a partition
- **Consumer group**: each partition read by one member
- Messages stay after reading (retention)

TERMINAL

```
$ pip install confluent-kafka
$ docker run -d -p 9092:9092 apache/kafka
```

Kafka Producer with confluent-kafka

kafka_producer.py

```python
import json

from confluent_kafka import Producer

producer = Producer({"bootstrap.servers": "localhost:9092",
                     "acks": "all", "enable.idempotence": True})


def on_delivery(err, msg) -> None:
    if err is not None:
        print("delivery failed:", err)
    else:
        print(f"{msg.topic()}[{msg.partition()}] offset {msg.offset()}")


event = {"type": "OrderPlaced", "order_id": 17, "customer_id": 42}
producer.produce("orders", key=str(event["customer_id"]),
                 value=json.dumps(event), on_delivery=on_delivery)
producer.flush()                          # wait for all deliveries
```


Kafka Consumer with confluent-kafka

kafka_consumer.py

```python
import json

from confluent_kafka import Consumer, KafkaException

consumer = Consumer({"bootstrap.servers": "localhost:9092",
                     "group.id": "shipping", "enable.auto.commit": False,
                     "auto.offset.reset": "earliest"})
consumer.subscribe(["orders"])
try:
    while True:
        msg = consumer.poll(timeout=1.0)
        if msg is None:
            continue
        if msg.error():
            raise KafkaException(msg.error())
        event = json.loads(msg.value())
        print(f"p{msg.partition()}@{msg.offset()}: ship {event['order_id']}")
        consumer.commit(message=msg, asynchronous=False)   # after work
finally:
    consumer.close()                    # leave the group cleanly
```

6 · MESSAGING IN PRACTICE

Delivery Guarantees

Guarantee	Consumer acknowledges…	After a crash	Typical use
At-most-once	**before** processing (auto-ack)	message may be **lost**	metrics, sensor readings
At-least-once	**after** processing	message may be **duplicated**	most business events
Exactly-once	in the same transaction as the result	neither — at a cost	payments, ledgers

Kafka

Exactly-once *within Kafka*: idempotent producer + transactions (read–process–write)

RabbitMQ

At-most-once (auto-ack) or at-least-once (manual ack + publisher confirms)

In practice

End-to-end: at-least-once delivery + **idempotent** consumers

6 · MESSAGING IN PRACTICE

Idempotent Consumers

consumer.py

```python
class PaymentConsumer:
    """Applies each payment at most once."""

    def __init__(self) -> None:
        self.seen: set[str] = set()
        self.balance = 0.0

    def handle(self, event: Payment) -> None:
        if event.event_id in self.seen:
            print("skip duplicate", event.event_id)
            return
        self.balance += event.amount
        self.seen.add(event.event_id)
```

`Payment` is a frozen dataclass: `event_id`, `order_id`, `amount`.

```python
consumer = PaymentConsumer()
payment = Payment("evt-1", order_id=17,
                  amount=249.9)
for delivery in (payment, payment):
    consumer.handle(delivery)  # redelivery
print(consumer.balance)
```

OUTPUT

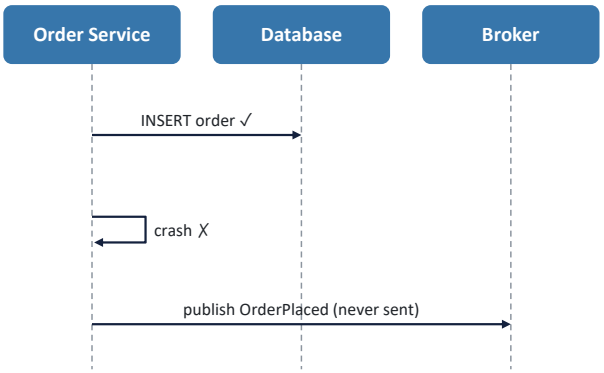
```
skip duplicate evt-1
249.9
```

Key idea

Idempotent: processing twice has the same effect as once.

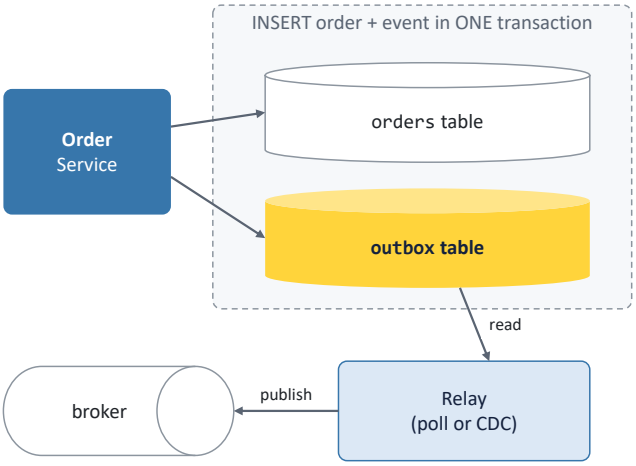
Dual Writes and the Transactional Outbox

Problem: two writes, no shared transaction



The order exists, but no event is ever published.

Solution: write the event to an outbox table



Outbox in Python: One Transaction

outbox.py

```python
import json
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, total REAL)")
db.execute("CREATE TABLE outbox (id INTEGER PRIMARY KEY, "
           "payload TEXT, sent INTEGER DEFAULT 0)")


def place_order(total: float) -> None:
    """Store the order AND its event in one transaction."""
    with db:                                  # commit both or neither
        cur = db.execute("INSERT INTO orders (total) VALUES (?)",
                         (total,))
        event = {"type": "OrderPlaced", "order_id": cur.lastrowid}
        db.execute("INSERT INTO outbox (payload) VALUES (?)",
                   (json.dumps(event),))
```

Key idea

`with db:` commits both INSERTs together, or rolls both back on an exception.

6 · MESSAGING IN PRACTICE

Outbox in Python: The Relay

relay.py

```python
from collections.abc import Callable


def relay(publish: Callable[[str], None]) -> None:
    """Publish unsent events, then mark them as sent."""
    rows = db.execute("SELECT id, payload FROM outbox WHERE sent = 0")
    for row_id, payload in rows.fetchall():
        publish(payload)                    # may repeat after a crash
        with db:
            db.execute("UPDATE outbox SET sent = 1 WHERE id = ?",
                       (row_id,))


place_order(249.9)
place_order(99.0)
relay(lambda payload: print("publish", payload))
relay(lambda payload: print("publish", payload))   # nothing left
```

OUTPUT

```
publish {"type": "OrderPlaced", "order_id": 1}
publish {"type": "OrderPlaced", "order_id": 2}
```


FIND THE BUG

Find the Bug: The Consumer That Loses Orders

```python
def on_message(channel, method, properties, body: bytes) -> None:
    order = json.loads(body)
    reserve_stock(order)
    charge_card(order)          # raises when the card service is down


channel.basic_consume("orders", on_message, auto_ack=True)
channel.start_consuming()
```

- When the card service is down, some orders are never charged
- The broker shows zero messages waiting in `orders`
- Which delivery guarantee does this consumer have? Fix it.

ANSWER

Find the Bug: The Consumer That Loses Orders

```python
def on_message(channel, method, properties, body: bytes) -> None:
    order = json.loads(body)
    try:
        reserve_stock(order)    # must be idempotent: redelivery!
        charge_card(order)
    except ConnectionError:
        channel.basic_nack(method.delivery_tag, requeue=True)
        return
    channel.basic_ack(method.delivery_tag)      # after the work


channel.basic_consume("orders", on_message)     # auto_ack=False
```

Key idea

`auto_ack=True` acknowledges on delivery: a failure after that loses the message (at-most-once). Ack after the work, and make the work idempotent.

QUIZ

Quick Check

A Kafka consumer commits each offset *before* processing the message. Which delivery guarantee does it provide?

- A At-most-once
- B At-least-once
- C Exactly-once
- D No guarantee at all

ANSWER

Quick Check

A Kafka consumer commits each offset *before* processing the message. Which delivery guarantee does it provide?

- **A** At-most-once
- **B** At-least-once
- **C** Exactly-once
- **D** No guarantee at all

Why

If it crashes after the commit but before processing, the restarted consumer continues after the committed offset: that message is never processed.

07

Apache Kafka vs RabbitMQ

Two brokers, two philosophies: a replayable log vs a smart router

- Architecture and messaging model
- Retention, performance and scaling
- Guarantees, routing, operations and use cases
- Decision guidance

Kafka vs RabbitMQ: Architecture and Model

	Apache Kafka	RabbitMQ
Positioning	distributed event-streaming platform	message broker (AMQP)
Core structure	partitioned, replicated commit log	exchanges, bindings and queues
Consumption	consumers **pull** and track offsets	broker **pushes** to consumers
After reading	message stays until retention expires	message removed once acknowledged

★ **Key idea**

Kafka remembers; RabbitMQ delivers and forgets.

Kafka vs RabbitMQ: Retention, Speed and Scale

	Apache Kafka	RabbitMQ
Retention and replay	time/size-based retention; rewind offsets to replay	classic queues: no replay; streams (3.9+) add it
Throughput	very high: sequential disk I/O, batching	high; optimised for low per-message latency
Scaling	add brokers and partitions	clustering, quorum queues; big queues are harder
Parallelism	≤ one consumer per partition per group	many competing consumers per queue

! **Watch out**

Throughput depends on message size, durability and replication settings — benchmark your own workload (Module 11), do not trust headline numbers.

Kafka vs RabbitMQ: Guarantees, Routing, Ops

	Apache Kafka	RabbitMQ
Delivery	at-least-once; exactly-once within Kafka	at-most-once or at-least-once
Ordering	per partition (choose keys well)	per queue (single consumer)
Routing	by topic and key; consumers filter	direct, topic, fanout, headers exchanges
Operations	plan partitions, replication, storage	simpler start, small–medium setups
Ecosystem	Connect, Streams, ksqlDB, Flink	AMQP 0-9-1/1.0, MQTT, STOMP
Typical uses	event sourcing, streaming, analytics	task queues, request-reply, jobs

Key idea

Order is only guaranteed per partition (Kafka) or per queue with one consumer (RabbitMQ): design keys and consumers for it.

Choosing Between Kafka and RabbitMQ

Choose Kafka when…

- Events must be **replayed** or kept for days
- Throughput is very high
- Many independent consumers read the same stream
- Event sourcing, streaming, analytics

Choose RabbitMQ when…

- Tasks must be **distributed** to workers
- Routing rules are rich or dynamic
- Per-message acks, priorities, TTLs matter
- The team wants a simpler start

Tip

Many organisations run both: Kafka as the event backbone, RabbitMQ for work queues.

QUIZ

Quick Check

An analytics team must re-run a new algorithm over the last 7 days of click events. Which option fits best?

- **A** A RabbitMQ classic queue with manual acks
- **B** A Kafka topic with 7-day retention
- **C** An in-memory event bus inside the web app
- **D** Synchronous REST calls to the web servers

ANSWER

Quick Check

An analytics team must re-run a new algorithm over the last 7 days of click events. Which option fits best?

- A A RabbitMQ classic queue with manual acks
- **B** A Kafka topic with 7-day retention
- C An in-memory event bus inside the web app
- D Synchronous REST calls to the web servers

Why

Kafka keeps the events after they are read; the team starts a new consumer group from the oldest retained offset and replays the week.

MODULE 09 · PRACTICE

Practice Problems

1. Classify

List 8 messages of a food-delivery app and label each as command, event or query

2. Temporal query

Add `CustomerRenamed` to the event-sourced `Customer`; print its name at every version

3. New read model

Add an `orders_by_customer` projection to the CQRS example

4. Robust bus

Extend `EventBus` with retries and a dead-letter list for failing handlers

5. Watch a redelivery

Run the `pika` producer and consumer; kill the consumer before `basic_ack` and watch the message return

6. Kafka or RabbitMQ?

Pick a broker for: IoT telemetry, invoice PDF jobs, a fraud model retrained nightly

BROKERS FOR PROBLEMS 5 AND 6 (DOCKER)

```
$ pip install pika confluent-kafka
$ docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:4-management
$ docker run -d -p 9092:9092 apache/kafka
```


MODULE 09 · SUMMARY

Key Takeaways

- ✓ Commands request, events announce, queries ask — names make the intent visible
- ✓ Event notification decouples senders from receivers but hides the overall flow
- ✓ Event-carried state transfer avoids call-backs, at the cost of eventually consistent replicas
- ✓ Event sourcing keeps every change; state is replayed from the log, snapshots make it fast
- ✓ CQRS separates write and read models — apply it only where the domain needs it
- ✓ Brokers deliver at-most- or at-least-once: use idempotent consumers and the outbox
- ✓ Kafka is a replayable, partitioned log; RabbitMQ is a flexible message router

NEXT UP Module 10 — Reactive Programming with RxPY